

Degrader Firmware

iThemba LABS beam-line degrader control

iThemba LABS Test Rig — FirmwareUpdate/ILT_OS/Applications/Degrader

September 11, 2026

Abstract

The degrader places absorber lenses in and out of the proton beam. This document describes the firmware that drives it: how it is layered, what runs on which thread, the network interface it presents, how it is configured and built, and where it departs from the original `ithemba_control_system/degrader` firmware it replaces.

Contents

1	What the firmware does	2
2	How the code is arranged	2
2.1	Files	3
3	Threads	3
3.1	Timings	3
4	State model	4
4.1	Lens state	4
4.2	Process status	4
4.3	The response word	4
5	Network interface	4
5.1	HTTP routes	4
5.2	Requesting a lens set	5
5.3	Mechanism state	5
5.4	The status stream	5
6	Configuration	6
6.1	Backwards compatibility	6
7	Building and flashing	7
7.1	Host build	7
7.2	Cross build	7
7.3	Flashing	7
8	Hardware interface	7
9	Indicators	8

10 Known limitations

8

1 What the firmware does

The mechanism holds seven absorber lenses of differing thickness — 2, 3, 6, 8, 10, 12 and 30 mm. Each can independently be in the beam or clear of it. A control station asks for a *set* of lenses; the firmware works out what has to move and drives the mechanism until the requested set is in place.

Moving one lens takes two motors:

1. the **stepper** rotates the carousel until the requested lens is at the working station, confirmed by that lens's **SELECT** switch;
2. the **DC motor** then drives that lens into or out of the beam, confirmed by its **IN** or **OUT** switch.

Every motion is confirmed by a limit switch. Nothing is driven open-loop and no position is assumed — if a switch does not close within the move timeout the move is abandoned and the mechanism is reported faulted.

2 How the code is arranged

The firmware is three layers. The rule that keeps them apart is worth stating plainly, because it is what lets the same application run on more than one board:

Rule Nothing under `ILT_OS/Lib` or `ILT_OS/Applications` may include a chip header. Everything an application knows about the hardware arrives through `BSP/Include/Bsp/*.h`.

Layer	Responsibility
<code>BSP/Boards/<board></code>	The chip. HAL initialisation, pin mapping, timers, Ethernet. One directory per board; selected with <code>-DILT_BOARD</code> .
<code>ILT_OS/Lib</code>	Portable services over FreeRTOS and lwIP: Thread, Mutex, Queue, Timer, NetworkStack, HttpServer, StreamServer.
<code>ILT_OS/Applications/Degrader</code>	This firmware. The mechanism sequence and the network interface.

Table 1: The three layers, outermost last.

Exactly one application is linked into an image, chosen with `-DILT_APP`. The application is reached through a single entry point:

```
extern "C" void ILT_ApplicationStart(void)
{
    bsp::boardInit();
    g_supervisor.start();
}
```

Listing 1: The only symbol the system requires of an application.

2.1 Files

File	Contains
DegraderApp.cpp	Supervisor thread, network bring-up, HTTP routes, the status stream.
Degrader.h/.cpp	The mechanism itself: the move sequence, lens state, and the packed status word.
IndexPage.h	The built-in control page, a string literal in flash.
CMakeLists.txt	Configuration options and the <code>ilt_app</code> target.

`Degrader.h/.cpp` are shared verbatim with the combined `ILT_TESTRIG` image, so behaviour cannot drift between the two.

3 Threads

Two threads run, plus lwIP's own `tcpip_thread`.

Thread	Priority	Does
<code>degrader-sup</code>	Normal	Brings the network up, registers routes, then every 250 ms updates the LEDs and publishes the status word if it changed.
<code>degrader</code>	Normal	Waits on a request queue and runs the move sequence.

The mechanism thread is written as straight-line code. The original firmware ran an explicit six-state machine stepped from a superloop, with busy-waits spelled as `while (HAL_GetTick() - t < 5) {}`. On a thread the states disappear, because the program counter already remembers where the sequence has got to, and the waits become sleeps that yield the CPU.

3.1 Timings

Constant	Value	Meaning
<code>kSettleMs</code>	5 ms	Pause after a motor stops, before reading a switch.
<code>kSwitchPollMs</code>	2 ms	Interval while waiting for a switch to close.
<code>kMoveTimeoutMs</code>	30 000 ms	Longest a single move may take before it is failed.
<code>kIdlePollMs</code>	100 ms	Interval while no move is outstanding.

Note The move timeout is the one behaviour that is genuinely new. The original firmware had no timeout: a switch that never closed hung the sequence indefinitely. Here the move fails, `moveFailures()` increments and the mechanism reports `Fault`.

4 State model

4.1 Lens state

Each lens is one of four states. The numeric values are the two-bit encoding the original response word used, so a control station reading the packed word sees what it has always seen.

Value	State	Meaning
0	Out	Clear of the beam.
1	In	In the beam.
2	Moving	Being driven; neither in nor out.
3	Unknown	Not yet established, e.g. before the first move.

4.2 Process status

Value	Status	Meaning
0	Processing	A move is in progress; new requests queue behind it.
1	Awake	Powered but not yet homed.
2	Fault	A move timed out, or there is no mechanism attached.

4.3 The response word

`responseWord()` packs the seven lens states into one 16-bit value, two bits per lens, lens 0 in the least significant pair. It is the single value that changes whenever anything about the mechanism changes, which is why it is what the status stream watches.

5 Network interface

The original firmware owned a TCP server for commands and a TCP client that dialled out with data. This firmware splits that by shape:

- **requests** — HTTP on port 80, one path per operation, JSON replies;
- **state** — one TCP port that clients connect to and stay connected to, receiving a JSON object per line as the mechanism changes.

A control station therefore needs no reconnect-per-sample and no bespoke binary framing, and `curl` or a browser is enough to drive the rig by hand.

5.1 HTTP routes

Five routes, inside the default `ILT_HTTP_MAX_ROUTES` of 12.

Route	Returns
/index.html	The built-in control page.
/api/status.json	Board, address, uptime, link, stream counters.
/api/degrader.json	Full mechanism state.
/api/degrader/set.json	Requests a lens set. Takes ?mask=0..127.
/favicon.ico	204 No Content.

5.2 Requesting a lens set

The mask is a bit per lens, bit 0 being the 2 mm lens:

```
$ curl 'http://192.168.7.104/api/degrader/set.json?mask=5'
{"ok":true,"mask":5}

$ curl 'http://192.168.7.104/api/degrader/set.json?mask=999'
{"ok":false,"mask":999}

$ curl 'http://192.168.7.104/api/degrader/set.json'
{"ok":false,"error":"expected ?mask=<0..127>"}
```

Listing 2: Put the 2 mm and 6 mm lenses in the beam.

A mask above 0x7F is rejected rather than truncated.

5.3 Mechanism state

```
{"present":false,"status":2,"desired":0,"current":0,
 "word":49151,"moves":0,"failures":0,
 "motor":{"motion":"idle","target":-1,"confirmed":-1,
  "steps":0,"milli_stations":0,
  "steps_per_station":200,"step_rate_hz":200,"stations":7}}
```

Listing 3: /api/degrader.json, as measured on a board with no mechanism attached.

`present` is false when the board has no degrader hardware, which is how a NUCLEO development board reports itself. `motion` is one of `idle`, `selecting`, `inserting` or `retracting`.

5.4 The status stream

A client connects to the stream port and stays connected. One JSON object per line arrives whenever the status word or the process status changes, and once immediately on connect.

```
{"type":"degrader","t_ms":184775,"status":2,"word":49151,
 "desired":0,"current":0,"motion":"idle","moves":0,"failures":0}
```

Listing 4: A record from the stream.

Why The record on connect exists because on-change alone is not enough. The first publication happens at boot, long before anyone connects, so a client attaching to a rig that is simply sitting still would see an open socket and no data — and on a rig with no mechanism attached, nothing would ever change to break the silence. This was found on hardware: the socket was accepted and `sent` stayed at 0.

Unlike the original, the board does not dial out. A client connects to it, which removes the reconnect-per-sample the old client performed and means the rig does not need to know the control station’s address.

6 Configuration

Options are CMake cache variables, set at configure time:

```
cmake --preset Debug -DILT_APP=Degrader -DDEGRADER_HOSTNAME=degrader-1
```

Variable	Default
DEGRADER_STACK_BYTES	1536
DEGRADER_HOSTNAME	ilt-degrader
DEGRADER_STREAM_PORT	1971
DEGRADER_USE_DHCP	OFF
DEGRADER_DHCP_TIMEOUT_MS	15000
DEGRADER_FALLBACK_IP	192.168.7.104
DEGRADER_FALLBACK_NETMASK	255.255.255.0
DEGRADER_FALLBACK_GATEWAY	192.168.7.1

Table 2: Configuration, with the values that preserve the original firmware’s addressing.

The dotted quads are validated at configure time and a stream port outside 1–65535 is rejected, so a typo fails the configure rather than the boot.

6.1 Backwards compatibility

The defaults are taken from `ithemba_control_system/degrader`:

Original	Value	Here
LOCAL_IP	192.168.7.104	fallback address
SERVER_SEND_PORT	1971	stream port
SERVER_RECV_PORT	1970	<i>not preserved</i>
SERVER_IP	192.168.7.1	gateway
addressing	static	DHCP off

Caution The command port is **not** backwards compatible. The original listened on 1970 and spoke a binary framing with a 0xA5 sync byte; requests here are HTTP on port 80. A control station pointed at 1970 will not find a listener, and would not understand the reply if it did. Only the addressing and the data port are preserved.

7 Building and flashing

7.1 Host build

The application can be built and run natively, which is the fastest way to check that it compiles and starts. Hardware is simulated and networking runs over a TAP interface.

```
cd FirmwareUpdate
cmake -S . -B build/HostDeg -G Ninja \
      -DILT_BOARD=Host -DILT_APP=Degrader
cmake --build build/HostDeg
./build/HostDeg/FirmwareUpdate
```

7.2 Cross build

```
cmake -S . -B build/F767_Degrader -G Ninja -DCMAKE_BUILD_TYPE=Debug \
      -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi-gcc.cmake \
      -DILT_BOARD=NucleoF767ZI -DILT_APP=Degrader
cmake --build build/F767_Degrader
```

Measured footprint on the F767: 99.4 kB of flash and 97.1 kB of RAM, against 2 MB and 512 kB available.

7.3 Flashing

```
STM32_Programmer_CLI -c port=SWD mode=UR \
  -w build/F767_Degrader/FirmwareUpdate.hex -v -rst
```

The console is USART3 on the ST-LINK virtual COM port at 115 200 baud. A healthy boot reads:

```
Degrader on NUCLEO-F767ZI
rig: no degrader hardware on this board
http: listening on port 80
stream: listening on port 1971
net: ready -- http://192.168.7.104/ and stream on port 1971
```

8 Hardware interface

Everything the application knows about the mechanism comes through `Bsp/Degrader.h`:

```
bool    available() noexcept;
void    init() noexcept;
bool    readSwitch(Lens lens, LimitSwitch which) noexcept;
void    stepperStart(StepperDirection direction) noexcept;
void    stepperStop() noexcept;
std::uint32_t stepsIssued() noexcept;
std::int32_t stepPosition() noexcept;
```

`available()` is what makes the firmware safe to run on a board with no mechanism: it reports false, no motor is driven, and the status is `Fault` rather than a crash.

9 Indicators

LED	Meaning
Status	Solid once addressed; blinking while it is not.
Activity	On while the mechanism is moving or a move has just completed.
Fault	On when the mechanism reports <code>Fault</code> .

The board's state is therefore readable without a serial terminal.

10 Known limitations

1. **The command protocol is not backwards compatible.** See §6.1. Restoring it means re-implementing the binary server against the `Bsp` layer and a second listener port.
2. **The HTTP port is fixed at 80.** It is lwIP's `httpd`, set board-wide in `lwipopts.h` rather than per application.
3. **Three added stepper drivers are not driven.** The custom board carries three further A4988s; no `Bsp` entry point exists for them yet, and one of the three `STEP` lines cannot be hardware PWM. See the board README.